CS2201: Computer Organization & Architecture – One-Page Progress Report

Oral presentation to assigned TA (Oct 28 – Oct 30, 2026) | Final assignment deadline: Nov 08, 2026

Group: GN4 (Coding Assignment Track) **Submission file:** CS2201_C_GN4_X7 **TA:** Nikhil (n.khatuwal@iitg.ac.in)
Paper X7: *From Optimal to Practical: Efficient Micro-op Cache Replacement Policies for Data Center Applications* (Kan Zhu, Yilong Zhao, Yufei Gao, Peter Braun, Tanvir Ahmed Khan, Heiner Litz, Baris Kasikci, Shuwen Deng)

1. Group members

Member	Name	Roll no.	Part we are handling
M1	Abhishek Kumar (group rep)	250101001	Coordination with the TA, problem formulation, FLACK offline solver
M2	Kaustav Bhowal	250101050	7-step profiling pipeline, Jenks natural breaks (3-bit grading)
M3	Jayesh Bansal	250101046	FURBYS hardware replacement, selective bypass, pitfall detector

2. Problem statement and why it matters

Modern x86 processors have a small micro-op cache (we model AMD Zen3: 512 entries, 8-way set-associative, up to 8 micro-ops per entry). It hands decoded micro-ops straight to the backend, so the L1 instruction cache and the decoders can be powered down. Data center applications have huge instruction footprints, and with LRU more than 99% of the misses are either capacity misses (88.31%) or conflict misses (10.80%). The policies we compared against (LRU, SRRIP, SHiP++, GHRP, Belady's MIN and the flow-based offline optimal, FOO) all do worse than they could, mainly because they ignore three things specific to this cache: (1) prediction windows (PWs) take up different numbers of entries and hold anywhere from 1 to 8 valid micro-ops, so size and cost vary; (2) partial hits are possible when PWs share a start address but differ depending on whether a conditional branch is taken; and (3) lookups and insertions are asynchronous, since the non-blocking accumulation buffer delays insertion.

3. What we have done so far

A. Reading and design

- **FLACK (offline):** the paper improves on FOO in three ways. It scales the miss penalty by cost/size (valid micro-ops per entry), it keeps bypassed PWs so they can still give partial hits, and it buffers evictions lazily to deal with the delayed insertions.
- **FURBYS hardware cost:** for one 8-way set we get (3-bit weight + 2-bit RRPV) × 8 ways + 2 × 3-bit pitfall detector = 46 bits, which is under 1% of the 4,608 data bits in a set. We verified this calculation ourselves.
- **Decoder and accumulator:** traced how the 3-bit Jenks weight hints, stored in reserved branch bits, move through the decoder and the accumulation unit into the micro-op cache.

B. Simulator and code

- **Trace profiler (step 2):** records the PW lookup sequence with the micro-op cache size set to 0. The offline FLACK cost-aware hit-rate calculation (steps 3–5) is also done.
- **Jenks natural breaks (step 6):** dynamic-programming implementation that splits the per-set PW hit rates into 8 clusters, which become the 3-bit weights 0 to 7.
- **FURBYS engine (step 7):** 8-way micro-op cache written in Python (furbys_uop_cache_sim.py) and as a C++ Scarab module (uop_cache_furbys.cc). It does selective bypassing (when w_new < w_min - 1) and has a 2-slot local pitfall detector that falls back to 2-bit SRRIP if the same ways keep getting evicted.

4. Results so far and what is left before Nov 08

Policy	Metadata / set	Partial-hit aware	Cost/size aware	Sim. uop misses	Paper: avg. miss reduction, PPW gain
LRU (baseline)	24 bits	No	No	56,164 (baseline)	0.00% miss red., 0.00% PPW
SRRIP (2-bit RRPV, online)	16 bits	No	No	47,723 (-15.03% vs LRU)	~5.80% miss red., +0.45% PPW
GHRP (best online)	Predictor tables	No	No	Baseline benchmark	~9.52% miss red., +0.58% PPW
FURBYS (cross-input profile)	46 bits (<1%)	Yes (via FLACK)	Yes (3-bit Jenks)	26,144 (-53.45% vs LRU)	13.51% miss red. (94.34% of same-input)
FURBYS (same-input profile)	46 bits (<1%)	Yes (via FLACK)	Yes (3-bit Jenks)	24,669 (-56.08% vs LRU)	14.34% miss red., +3.10% PPW (1.5× cache)

Still to do (Oct 31 – Nov 08, 2026): finish the multi-trace sensitivity plots (varying the number of representative bits from 1 to 8 and the pitfall detector depth from 1 to 5), then submit the final report (CS2201_C_GN4_X7.pdf) along with the code package before Nov 08, 11:59 PM.